

Área Departamental de Engenharia de Eletrónica e Telecomunicações e de Computadores
Licenciatura em Engenharia Eletrónica e Telecomunicações e de Computadores

Projeto Final de Curso

Relatório de Progresso



Recetor de Rádio FM com Controlo Digital

João Santos, N°51009

Professor Doutor Vítor Fialho

6º Semestre letivo 2025/2026

16 de Abril de 2026

Índice

1. Introdução e Enquadramento	2
2. Adaptação do Hardware Inicial.....	2
3. Diagrama de Blocos do Projeto	3
4. Atividade Desenvolvida e Soluções Encontradas	4
4.1. Interface de Navegação e controlo	4
4.2. Controlo do Módulo RDA5807M	5
4.3. Algoritmo de Scan e Aquisição de Dados	6
4.4. Demonstração da Aquisição de Dados	6
5. Estado do Trabalho	7
5.1. Configuração do Hardware.....	7
5.2. Desenvolvimento do Firmware	7
5.3. Aquisição e Armazenamento de Sinais.....	7
5.4. Testes em diferentes locais	8
5.5. Desenvolvimento do algoritmo de classificação	8
6. Calendarização do trabalho.....	8
7. Referências.....	9

Índice de Figuras

Figura 1 - Arduino Nano ESP32.	2
Figura 2 - Radio FM RDA5807.	3
Figura 3 - Diagrama de Blocos do Projeto.....	3
Figura 4 - Breakout do módulo Radio FM RDA5807.	4
Figura 5 - 5-Way Navigation Switch.	4
Figura 6 - Código implementado para a utilização do joystick.	5
Figura 7 - Função Implementada para extração do nível de sinal (RSSI).....	5
Figura 8 - Implementação das funções de controlo do volume.	5
Figura 9 - Fluxograma da função ScanFrequencies().	6
Figura 10 - Resultados obtidos no teste de sintonização.....	6
Figura 11 – Protótipo físico do Projeto.	7
Figura 12 - Diagrama de Gantt para a organização do projeto.	8

Índice de Tabelas

Tabela 1 - Principais Diferenças entre os Microcontroladores	2
--	---

1. Introdução e Enquadramento

Devido à sua ampla cobertura territorial, potência e reduzido consumo energético, a radiodifusão FM continua a ter um papel crítico na propagação de informação e entretenimento para a população. Além disto, têm-se evidenciado uma grande vulnerabilidade nas infraestruturas modernas de comunicação, notavelmente durante apagões elétricos e situações de calamidade, como ocorrido recentemente em Portugal. Em cenários de emergência como estes, a análise da qualidade do sinal FM é estritamente necessária para avaliar a sua eficácia, que pode ser muito variável. A qualidade da receção do sinal é dependente de diversos fatores, como a intensidade do sinal recebido, o ambiente envolvente (e.g. densidade da vegetação, habitações, etc.) e o tipo de antena utilizada. Atualmente, a utilização de circuitos integrados tem permitido o desenvolvimento de rádios compactos de baixo custo e facilmente manipulados por microcontroladores.

2. Adaptação do Hardware Inicial

No relatório inicial foi identificado como um dos principais desafios o “Armazenamento de dados”, uma vez que o microcontrolador inicialmente previsto, ARD-NanoV4-3V, não possuía memória suficiente para utilizar as bibliotecas desejadas, muito menos para conseguir processar o código. Este, ao ligar, apresentava um loop infinito de reset, demonstrando que o Arduino não tinha memória suficiente para sequer iniciar. De forma a ultrapassar este problema, o microcontrolador foi substituído pelo Arduino Nano ESP32 [Figura 1].

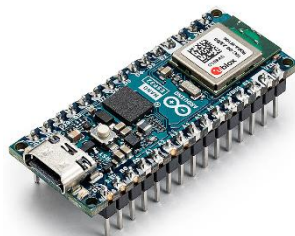


Figura 1 - Arduino Nano ESP32.

Este está equipado com um microprocessador Dual-core com um clock de 240MHz, 512KB de SRAM e uma memória flash de 16MB tendo ainda suporte para Wi-Fi 4 e Bluetooth LE, o que abre uma vasta gama de possibilidades futuras para o tratamento de dados. Com acesso ao Wi-Fi, é possível usar sistemas de cloud para armazenar os dados e o Bluetooth permite, por exemplo, o controlo remoto do rádio.

Em suma, as principais diferenças encontram-se apresentadas na tabela abaixo.

Tabela 1 - Principais Diferenças entre os Microcontroladores.

Microcontroladores	Clock	SRAM	Flash	Comunicações
ARD-NanoV4-3V	16MHz	2KB	32KB	Sem Wireless
Arduino Nano ESP32	240MHz	512KB	16MB	Wi-Fi e Bluetooth

Houve também uma mudança no Circuito Integrado baseado no RDA5807M. No relatório inicial tinha sido escolhido o Grove – I2C FM Receiver. Contudo, uma vez que este possui um breakout com os pinos Grove, a sua utilização revelou-se complicada, tendo sido decidido trocá-lo pelo Radio FM RDA5807 [Figura 2], desenvolvido pelo professor Pedro Sampaio. Este apresenta um breakout mais fácil de manusear, facilitando a sua utilização neste projeto.



Figura 2 - Radio FM RDA5807.

3. Diagrama de Blocos do Projeto

Conforme justificado no capítulo anterior, a necessidade de maior capacidade de processamento e memória motivou a substituição do microcontrolador originalmente previsto pelo Arduino Nano ESP32. Esta transição exigiu uma reformulação do esquema de ligações, refletida no diagrama de blocos atualizado [Figura 3]. Uma das grandes diferenças é a introdução do par de resistências de pull-up nas linhas SDA (Serial Data) e SCL (Serial Clock). Esta melhoria assegura que o barramento I2C partilhado opere com níveis lógicos de 3.3V estáveis, minimizando, assim, o ruído e possíveis falhas de comunicação entre o Arduino e os periféricos.

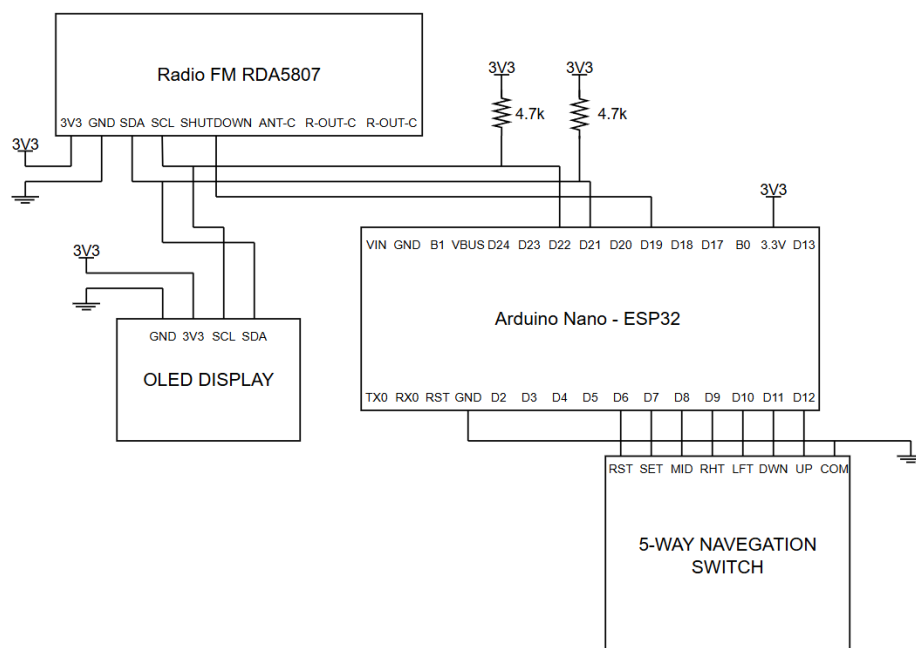


Figura 3 - Diagrama de Blocos do Projeto.

A *Figura 4* apresenta o esquema elétrico do módulo Radio FM RDA5807, concebido sob a forma de uma PCB desenvolvida pelo professor Pedro Sampaio. Este módulo tem como principal componente o circuito integrado RDA5807. O chip é responsável pela recepção e desmodulação do sinal FM, sendo controlado pelo microcontrolador (Arduino Nano ESP32) através do barramento de comunicação I2C. Como os sinais analógicos de saída do RDA5807 (L-OUT e R-OUT) têm uma potência baixa, o módulo incorpora um amplificador de áudio estéreo dedicado, o TPA6111A2DR. Os sinais passam por condensadores de acoplamento AC antes de entrarem neste amplificador.

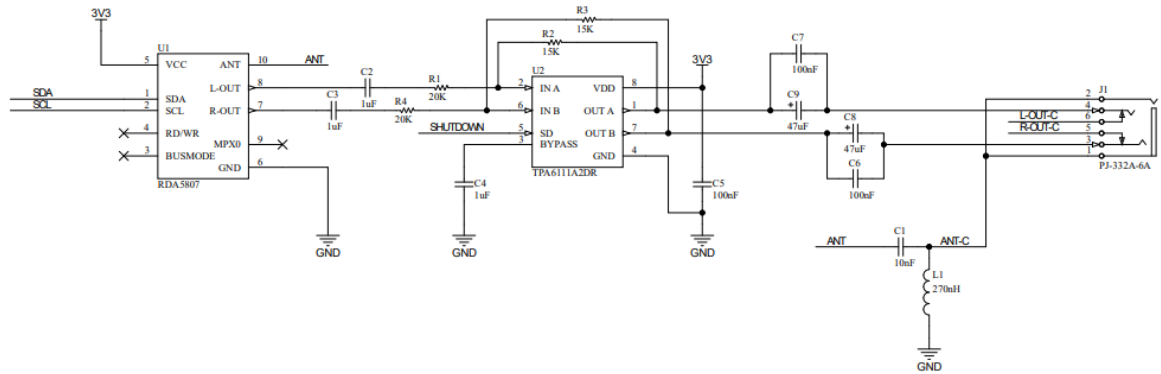


Figura 4 - Breakout do módulo Radio FM RDA5807.

4. Atividade Desenvolvida e Soluções Encontradas

Durante esta fase do projeto, o foco principal foi o desenvolvimento do firmware, assegurando a comunicação I2C entre o Arduino Nano ESP32, o módulo Radio FM RDA5807 e o Display OLED, bem como a implementação da lógica de controle.

4.1. Interface de Navegação e controlo

O controlo do rádio foi implementado com a utilização do 5-Way Navigation Switch da JOY-it [*Figura 5*]. Para otimizar a montagem, os pinos foram configurados no microcontrolador como *INPUT_PULLUP*. Desta forma, o sistema regista uma ação quando o nível lógico do pino transita para *low*. Os pinos UP e DOWN controlam o volume do amplificador interno do rádio, incrementando ou decrementando o valor dentro dos limites do chip RDA5807M (0 a 15). Já os pinos LEFT e RIGHT controlam a sintonização manual. Incrementa (RIGHT) ou decrementa (LEFT) a frequência em passos de 0.1MHz (100kHz). Por fim, o pino SET funciona para trocar o funcionamento do rádio, interrompe o funcionamento normal do rádio e lança a rotina *ScanFrequencies()*, que inicia o varrimento automático da banda e a recolha de amostras do valor de RSSI [*Figura 6*].



Figura 5 - 5-Way Navigation Switch.

```
int btn = ReadButton();

if (btn == BTN_UP) {
    RadioVolUp();
}
else if (btn == BTN_DWN) {
    RadioVolDown();
}
else if (btn == BTN_LEF) {
    frequency += 10;
    RadioSetFreq(frequency);
}
else if (btn == BTN_RHT) {
    frequency -= 10;
    RadioSetFreq(frequency);
}
else if (btn == BTN_SET) {
    ScanFrequencies();
}
```

Figura 6 - Código implementado para a utilização do joystick.

4.2. Controlo do Módulo RDA5807M

A interface de controlo e configuração do circuito integrado RDA5807 é realizada exclusivamente através do barramento I2C. Para apaziguar a complexidade da manipulação direta dos registos internos do recetor, o firmware desenvolvido recorre a uma biblioteca de controlo (Arduino Radio Lib) que gere o estado do dispositivo através da estrutura de dados *RADIO_INFO*.

Para a aquisição do nível de RSSI foi implementada uma função que atualiza a estrutura de informação do rádio e devolve o valor do RSSI num dado instante.

```
int getRSSI(){
    RADIO_INFO info;
    radio.getRadioInfo(&info);
    return info.rssi;
}
```

Figura 7 - Função Implementada para extração do nível de sinal (RSSI).

Já para o controlo do volume foram criadas duas funções *RadioVolUp()* e *RadioVolDown()*. Estas garantem que os comandos de incremento e decremento respeitam os limites do chip (de 0 a 15), prevenindo situações de possível overflow ou underflow no envio de comandos I2C.

```
void RadioVolUp() {
    if (currentVolume < 15) {
        currentVolume++;
        radio.setVolume(currentVolume);
        Serial.print("Volume: ");
        Serial.println(currentVolume);
    }
}

void RadioVolDown() {
    if (currentVolume > 0) {
        currentVolume--;
        radio.setVolume(currentVolume);
        Serial.print("Volume: ");
        Serial.println(currentVolume);
    }
}
```

Figura 8 - Implementação das funções de controlo do volume.

4.3. Algoritmo de Scan e Aquisição de Dados

Para o processo de medição de qualidade de sinal, foi criada a função *ScanFrequencies()*, ativada pelo botão SET do joystick. O algoritmo foi desenhado para percorrer a banda dos 80MHz aos 108MHz em incrementos de 100kHz. Para garantir a integridade dos dados, a cada alteração de frequência é imposto um tempo de espera de 100 milissegundos.

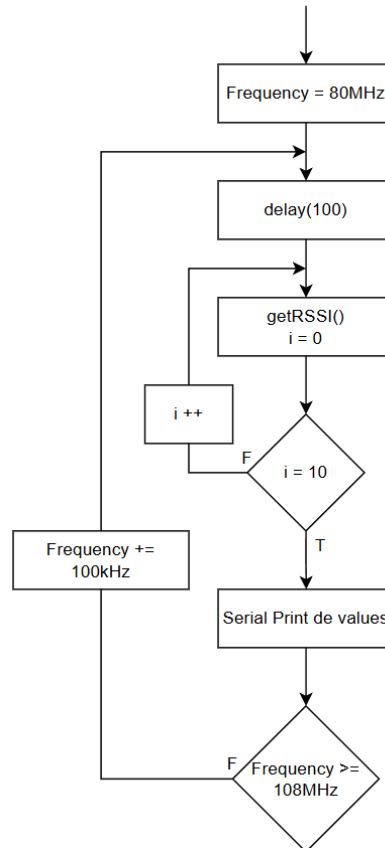


Figura 9 - Fluxograma da função *ScanFrequencies()*.

4.4. Demonstração da Aquisição de Dados

Para validar a implementação da rotina *ScanFrequencies()*, foram realizados testes de sintonização de sinal. O algoritmo foi executado com sucesso, percorrendo a banda e adquirindo 10 amostras do valor de RSSI por cada incremento de frequência.

Na Figura 10 apresenta-se um trecho dos resultados obtidos através do Serial Monitor do Arduino IDE. Importante salientar que este ensaio foi realizado no interior de um edifício.

```

Freq: 93.0 MHz | RSSI: 12, 13, 13, 14, 14, 14, 15, 15, 15, 15
Freq: 93.1 MHz | RSSI: 17, 19, 20, 20, 21, 22, 22, 22, 23, 23
Freq: 93.2 MHz | RSSI: 19, 20, 22, 23, 23, 24, 25, 25, 25, 26
Freq: 93.3 MHz | RSSI: 17, 18, 19, 20, 21, 21, 22, 22, 22, 23
Freq: 93.4 MHz | RSSI: 12, 13, 14, 14, 15, 15, 15, 15, 15, 16
  
```

Figura 10 - Resultados obtidos no teste de sintonização.

Ao analisar o trecho é possível detetar na frequência 93.2MHz (correspondente à emissora RFM) uma subida no indicador de força do sinal (RSSI). Apesar de o valor sofrer uma atenuação por este testes ter sido realizado dentro de um edifício, a discrepância face às frequências vazias é suficiente para classificar a presença de uma portadora válida. Em frequências afastadas do sinal útil, como a 90.3MHz e 93.44MHz, é nos devolvido um valor bastante consistente (entre os 12 e 15), assim, estes valores estabelecem o patamar do ruído. Por fim, é possível ainda observar que a transição do ruído para o sinal não é abrupta, pois, nas frequências imediatamente adjacentes (93.1MHz e 93.3MHz), o RSSI regista valores intermédios (de 17 a 23).

Em suma, estes resultados comprovam que tanto o hardware quanto o firmware estão a operar em perfeitas condições.

5. Estado do Trabalho

Tendo em consideração o Diagrama de Gantt estipulado no Relatório Inicial, o projeto encontra-se na seguinte situação de desenvolvimento:

5.1. Configuração do Hardware

Concluído. A transição para o microcontrolador Arduino Nano ESP32 e a integração do novo módulo Radio FM RDA5807 estabilizaram a arquitetura física do projeto. Este [Figura 11] encontra-se a operar consistentemente, pelo que a topologia de hardware é considerada finalizada.

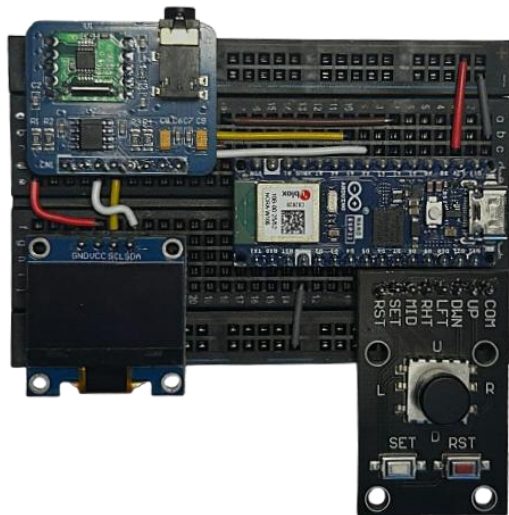


Figura 11 – Protótipo físico do Projeto.

5.2. Desenvolvimento do Firmware

Em fase avançada. As rotinas de comunicação I2C, controlo do rádio, leitura de RSSI e controlo do display estão plenamente funcionais. Agora o foco atual é a gestão de memória.

5.3. Aquisição e Armazenamento de Sinais

A fase de aquisição está operacional (com a rotina *ScanFrequencies()*). O armazenamento neste momento não existe, os valores de RSSI estão apenas a ser escritos no Serial Monitor do Arduino IDE, sendo a ideia principal, conseguir enviar os valores de RSSI para o portátil para depois os tratar de forma mais precisa.

5.4. Testes em diferentes locais

Pendente. A iniciar após completar o armazenamento dos valores de RSSI.

5.5. Desenvolvimento do algoritmo de classificação

Fase Inicial. A recolha das 10 amostras por canal já fornece os dados necessários para este algoritmo classificar a validade de uma estação, sendo necessário mais sinais e/ou amostras para classificar melhor uma estação.

6. Calendarização do trabalho

O Diagrama de Gantt presente [Figura 12] reflete o estado real da execução do projeto. Apesar do imprevisto do microcontrolador, todas as tarefas de desenvolvimento encontram-se em bom ritmo, permitindo que o projeto se mantenha alinhado com o previsto inicialmente.

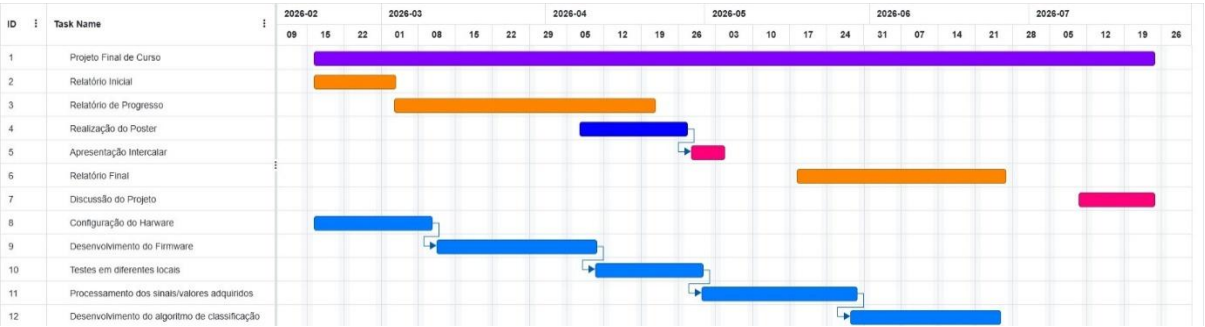


Figura 12 - Diagrama de Gantt para a organização do projeto.

7. Referências

[1] RDA5807M datasheet, “Single-Chip Broadcast FM Radio Tuner”, May 2011

https://cdn-shop.adafruit.com/product-files/5651/5651_tuner84_RDA5807M_datasheet_v1.pdf

[2] ARD-NanoV4-3V datasheet, 2025-06-26

https://joy-it.net/files/files/Produkte/ARD-NanoV4-3V/ARD-NanoV4-3V_Datasheet-EN_2025-06-26.pdf

[3] ARD-NanoV4-3V manual, 2025-06-26

https://joy-it.net/files/files/Produkte/ARD-NanoV4-3V/ARD-NanoV4-3V_Manual-EN_2025-06-26.pdf

[4] Arduino Nano ESP32, User Manual, 2026-4-14

<https://docs.arduino.cc/resources/datasheets/ABX00083-datasheet.pdf>